

IF25-22017 PENGEMBANGAN APLIKASI MOBILE

Pertemuan 11

Sprint 1: Perencanaan, Arsitektur, dan CI

Fase proyek dimulai: rencana kelompok, lapisan ui, domain, dan data, lalu GitHub Actions yang memeriksa setiap push

Muhammad Habib Algifari, S.Kom., M.T.I.

Tahun 2026



Program Studi Teknik Informatika
Fakultas Teknologi Industri
Institut Teknologi Sumatera

Tiga jawaban dari tiket keluar minggu lalu

Pertanyaan	Jawaban singkat
Apa itu fake, dan kenapa dipakai di unit test?	Implementasi tiruan yang cepat dan bisa diatur, sehingga test tidak butuh jaringan
Kenapa ViewModel lebih mudah diuji daripada composable?	ViewModel kelas biasa tanpa UI, cukup panggil fungsi lalu periksa state
Apa yang berubah di layar saat dependensi disediakan Koin?	Layar tidak berubah, hanya cara memperoleh ViewModel

Di laptop saya jalan kok

Kode kelompok sudah besar dan dikerjakan bertiga. Bagaimana memastikan setiap push tetap bisa di-build dan semua test lulus, tanpa mengandalkan laptop salah satu anggota?

DISKUSI 2 MENIT

Dimas yakin kodenya benar karena jalan di laptopnya. Ayu menarik kode terbaru dan build-nya gagal. Siapa yang salah, dan bagaimana kita tahu lebih cepat?

Terminal di laptop Ayu, Senin pagi

```
$ git pull
$ ./gradlew :composeApp:assembleDebug
e: LayarDaftar.kt:18 Unresolved reference 'ambilSemua'
BUILD FAILED in 38s

$ git log --oneline -3
a91f2c3 Ubah Sumber jadi pantauSemua
7d03e1b Tambah layar profil
c55b8a0 Perbaiki form laporan
```

Saat pulang nanti, kalian bisa

1 Menata lapisan
Memisahkan paket ui, domain, dan data dengan arah dependensi yang jelas.

2 Menjalankan CI
Membuat workflow GitHub Actions yang build dan test di setiap push dan pull request.

3 Bekerja sebagai tim
Memakai branch, pull request, dan review, lalu merencanakan sprint sampai minggu 16.

Capaian ini mendukung **CPMK0501**: menerapkan konsep pemrograman untuk pengembangan perangkat lunak, dalam bentuk aplikasi berlapis yang di-build dan diuji otomatis.

Bagian 1

Arsitektur berlapis

Setiap kelas punya rumah, dan setiap import hanya boleh ke arah tertentu.

Layar yang tahu terlalu banyak

ui/LayarDaftar.kt, versi tergesa-gesa

```
@Composable
fun LayarDaftar() {
    var daftar by remember {
        mutableStateOf(listOf<Laporan>())
    }
    LaunchedEffect(Unit) {
        val client = HttpClient {
            install(ContentNegotiation) { json() }
        }
        daftar = client.get("$BASE_URL/laporan").body()
    }
    LazyColumn {
        items(daftar) { KartuLaporan(it) }
    }
}
```

Kode ini jalan dan sering muncul di jawaban AI. Satu composable sekaligus membuat HttpClient, memanggil server, menyimpan state, dan menggambar daftar.

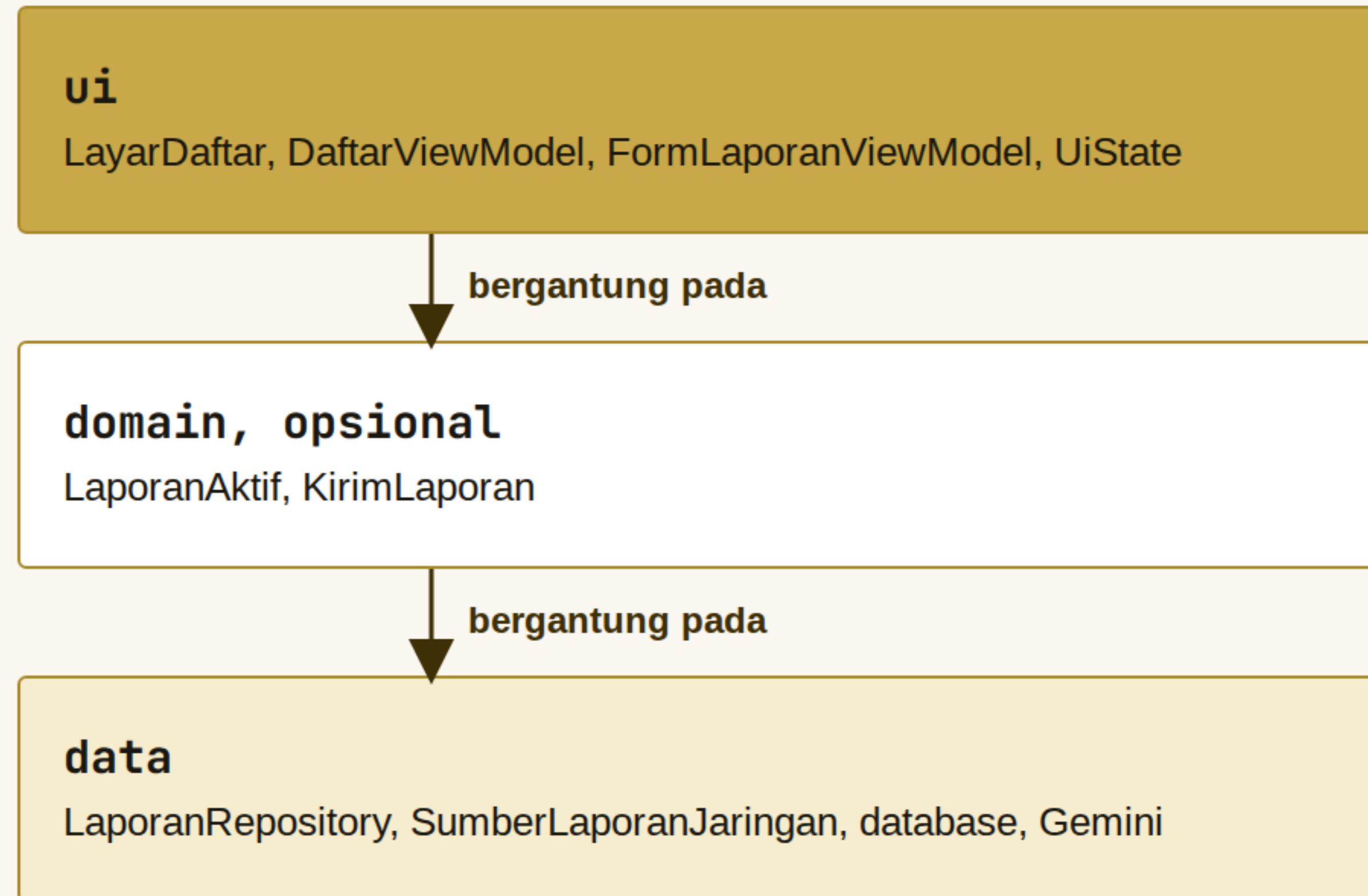
Bayangkan pola yang sama ditulis di lima layar wajib Bank Proyek.

BANGUN BARENG

Server Laporkampus berganti alamat dan nama field JSON

Kalau kelima layar ditulis seperti tadi, berapa berkas yang harus diubah? Dan bagaimana menguji LayarDaftar tanpa internet?

Tiga lapisan Laporkampus



Isi tiap lapisan

ui menampilkan dan menerima aksi. domain menyimpan aturan yang dipakai ulang. data tahu dari mana laporan datang: server, database, Gemini, atau sensor.

Lapisan domain boleh tidak ada. Panduan arsitektur Android menyebutnya opsional, dipakai hanya kalau logikanya rumit atau dipakai ulang.

Satu lapisan, satu paket

commonMain/kotlin

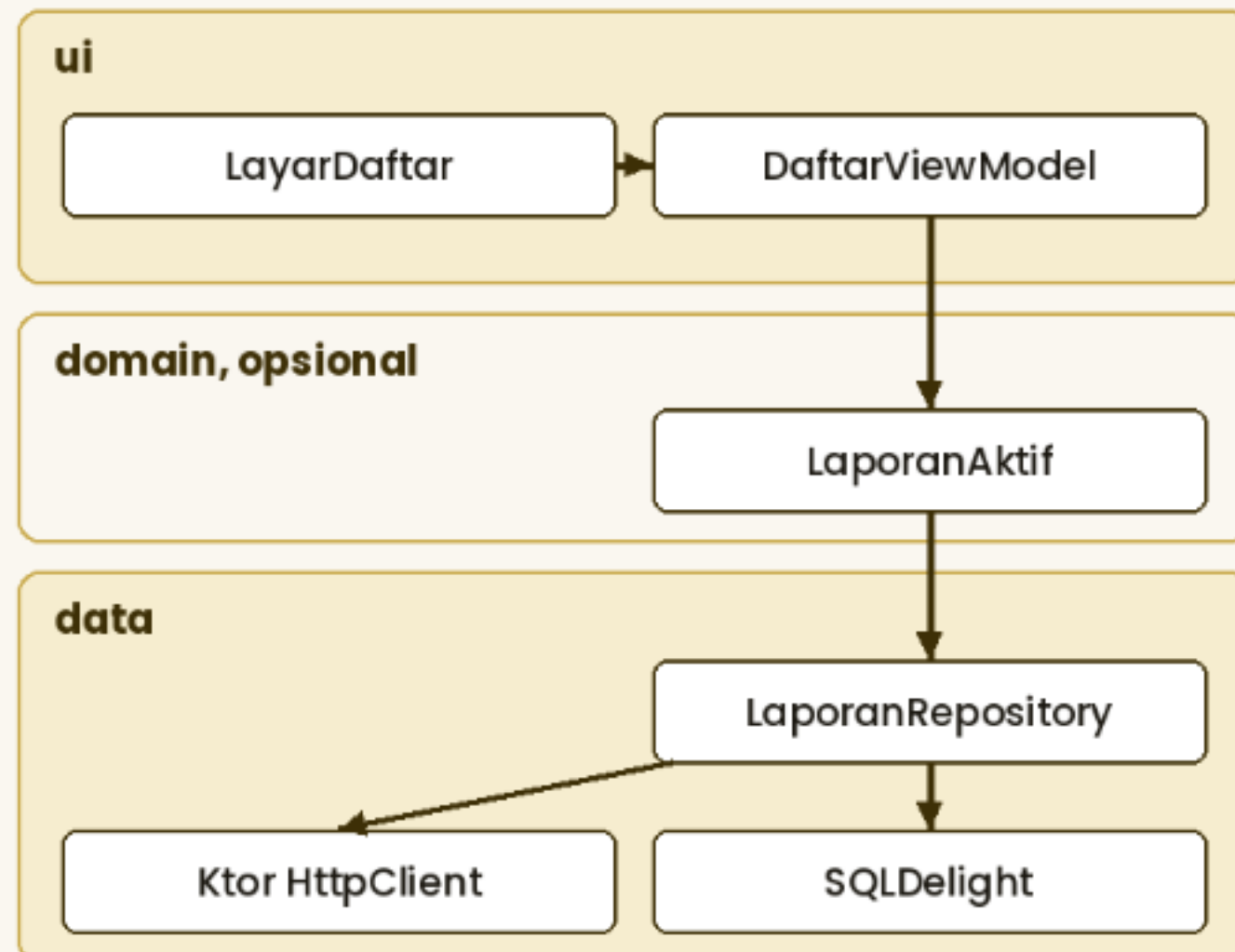
```
laporkampus/  
  App.kt  
  di/      ModulAplikasi.kt  
  ui/  
    daftar/  LayarDaftar.kt, DaftarViewModel.kt  
    detail/  LayarDetail.kt, DetailViewModel.kt  
    form/    FormLaporanScreen.kt, FormLaporanViewModel.kt  
    asisten/ LayarAsisten.kt, AsistenViewModel.kt  
  domain/   LaporanAktif.kt, KirimLaporan.kt  
  data/  
    model/   Laporan.kt, StatusLaporan.kt  
    remote/  SumberLaporanJaringan.kt, GeminiApi.kt  
    local/   LaporanLokal.kt  
  LaporanRepository.kt
```

Paket	Boleh mengimpor
ui	domain, data/model, Compose
domain	data
data	Ktor, SQLDelight, data/model
di	Semuanya, karena di sinilah Koin merakit

Ukurannya mudah: buka berkas di paket **ui**, lalu cari baris **import io.ktor**. Harusnya tidak ada.

Aturan dependensi

Aturan dependensi antarlapisan



Panah hanya ke bawah.
Lapisan bawah tidak tahu
lapisan atas.

Layar tidak pernah memanggil
Ktor, SQLDelight, atau Gemini
secara langsung.

Yang terjadi

Kalau aturan ini dijaga, setiap lapisan
bisa diuji dan diganti sendiri.

COBA LANGSUNG



Scan atau klik untuk membaca
panduan resmi lapisan UI,
domain, dan data, lengkap
dengan diagramnya.

[Guide to app architecture,](#)
[Android Developers](#)

Sumber: Dokumentasi Android Developers. Prinsipnya
sama untuk Kotlin Multiplatform.

Animasi berputar saat presentasi. Di PDF yang tampil keadaan akhirnya.

Use case, kalau memang perlu

commonMain/domain/LaporanAktif.kt

```
class LaporanAktif(  
    private val repo: LaporanRepository  
) {  
    operator fun invoke(): Flow<List<Laporan>> =  
        repo.pantauSemua().map { daftar →  
            daftar  
                .filter { it.status ≠ StatusLaporan.Selesai }  
                .sortedByDescending { it.prioritas }  
        }  
}
```

Keadaan	Perlu use case?
Aturan yang sama dipakai dua ViewModel	Ya
Menggabungkan dua repository	Ya
Hanya meneruskan pantauSemua()	Tidak, pakai repository langsung

operator fun invoke membuat objeknya bisa dipanggil seperti fungsi: `laporanAktif()`.

Layar daftar setelah dipisah

commonMain/ui/daftar

```
// ui/daftar/DaftarViewModel.kt
class DaftarViewModel(laporanAktif: LaporanAktif) : ViewModel() {
    val daftar = laporanAktif().stateIn(
        viewModelScope,
        SharingStarted.WhileSubscribed(5000),
        emptyList()
    )
}

// ui/daftar/LayarDaftar.kt
@Composable
fun LayarDaftar(vm: DaftarViewModel = koinViewModel()) {
    val daftar by vm.daftar.collectAsState()
    LazyColumn { items(daftar) { KartuLaporan(it) } }
}
```

Tambahan di Koin

```
// di/ModulAplikasi.kt
factory { LaporanAktif(get()) }
viewModel { DaftarViewModel(get()) }
```

LayarDaftar tidak lagi mengenal HttpClient, URL, atau JSON. Ganti server cukup di lapisan data.

Bagian 2

CI dengan GitHub Actions

Mesin netral yang build dan menjalankan test setiap kali ada yang push.

Tiga istilah CI

Continuous integration

Perubahan digabung sering, dan setiap perubahan langsung diperiksa dengan cara yang sama untuk semua anggota.

Workflow

Berkas YAML di folder `.github/workflows` yang menyebut kapan dijalankan dan langkah apa saja.

Runner

Mesin sementara milik GitHub, misalnya `ubuntu-latest`, yang selalu mulai dari kosong.

Berkas workflow pertama

.github/workflows/ci.yml

```
name: CI
on: [push, pull_request]
jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: 17
          cache: gradle
      - name: Build
        run: ./gradlew :composeApp:assembleDebug
      - name: Test
        run: ./gradlew :composeApp:desktopTest
```

Baris	Artinya
on	Dijalankan setiap push dan pull request
runs-on	Mesin runner yang dipakai
checkout	Mengambil kode dari repository
setup-java	Memasang JDK 17, cache Gradle menyimpan unduhan
run	Perintah yang sama dengan di terminal kalian

BANGUN BARENG

Dimas push langsung ke main, dan satu test gagal di CI

Apa yang terlihat di GitHub? Apakah kodenya tetap masuk ke main, dan apa akibatnya bagi Ayu yang git pull sesudahnya?

Cek tebakanmu: CI di pull request

Setiap push dan pull request menjalankan CI

git push, ke-2

fitur/filter-status

commit:

Perbaiki validasi judul

kosong

Hasil

Semua langkah lulus. Teman satu kelompok tinggal me-review.

job build-test

runs-on: ubuntu-latest

- | | | |
|-----------------|---------------------|-------|
| ● checkout | ambil kode | lulus |
| ● setup Java 17 | JDK temurin | lulus |
| ● cache Gradle | pakai ulang unduhan | lulus |
| ● build | assembleDebug | lulus |
| ● test | desktopTest | lulus |

Pull request

ke branch main

CI lulus

Siap di-review

dicek otomatis, bukan laptop

COBA LANGSUNG



Scan atau klik untuk membaca dasar CI aplikasi Android: build dan test otomatis setiap ada perubahan.

[Set up continuous integration, Android Developers](#)

Sumber: Dokumentasi Android Studio, Android Developers.

Animasi berputar saat presentasi. Di PDF yang tampil keadaan akhirnya.

Badge dan membaca hasil CI

README.md

```
# LaporKampus
```

```
![CI](https://github.com/AKUN/REPO/actions/workflows/ci.yml/badge.svg)
```

Tab Actions

Daftar semua run, hijau atau merah, lengkap dengan commit yang memicunya.

Log langkah

Klik langkah yang gagal. Pesan error biasanya ada di bagian bawah log.

Ulangi di laptop

Jalankan perintah yang sama, `./gradlew :composeApp:desktopTest`, sebelum push lagi.

Di laptop lulus, di CI gagal

Gejala atau pesan	Penyebab	Perbaikan
<code>./gradlew: Permission denied</code>	gradlew tidak punya izin eksekusi di git, sering dari Windows	<code>git update-index --chmod=+x gradlew</code>
<code>./gradlew: No such file or directory</code>	Gradle wrapper tidak ikut di-commit	Commit gradlew dan folder gradle/wrapper
Android Gradle plugin requires Java 17	JDK di runner lebih lama	<code>java-version: 17</code>
Test fitur AI gagal karena kunci kosong	local.properties tidak ada di CI, dan memang tidak boleh ada	Test memakai fake, bukan Gemini sungguhan
Test kadang lulus, kadang gagal	Bergantung pada waktu asli atau urutan test	Fake waktu seperti hands-on Testing dan DI

Bagian 3

Alur kerja tim dan rencana sprint

Branch, pull request, review, dan rencana sampai Demo Day.

Satu fitur, satu branch

Terminal

```
git switch main
git pull
git switch -c fitur/filter-status

# kerjakan, lalu commit kecil-kecil
git add .
git commit -m "Tambah filter status di daftar"
git push -u origin fitur/filter-status

# buka pull request ke main di GitHub
```

Kebiasaan

Alasannya

Nama branch jelas	fitur/..., perbaiki/..., semua orang tahu isinya
Commit kecil	Pesan commit menjelaskan apa yang berubah
Tarik main sering	Konflik kecil lebih mudah dari konflik besar

Pull request yang enak di-review

Deskripsi pull request

Tambah filter status di daftar laporan

Apa: chip Baru, Diproses, Selesai di atas daftar.

Kenapa: petugas hanya ingin melihat laporan Baru.

Cara cek: buka Beranda, ketuk chip Baru.

Test: DaftarViewModelTest, tiga test baru.

Tangkapan layar: sebelum dan sesudah.

Isi	Gunanya
Kecil, satu tujuan	Bisa dibaca dalam sepuluh menit
Deskripsi	Reviewer tahu apa dan kenapa
Tangkapan layar	Perubahan tampilan terlihat tanpa menjalankan
Test yang lulus	Bukti fitur lama tidak rusak

Menjaga branch main

Aturan untuk main

Di Settings, bagian Branches, tambahkan aturan untuk main: wajib lewat pull request, wajib satu persetujuan, dan wajib status check CI lulus.

	Tanpa aturan	Dengan aturan
Push ke main	Siapa saja, kapan saja	Hanya lewat pull request
CI merah	Kode tetap masuk	Merge diblokir
Keputusan	Satu orang	Minimal satu teman me-review

Rencana sprint minggu 11 sampai 16

Minggu	Fokus kelompok	Yang dinilai
11	Rencana, arsitektur rapi, CI hijau	Checkpoint, bahan tes lisan 1
12	Fitur utama lengkap: lima layar, data REST, offline	Tes lisan 1, 10%
13	Integration test, performa, fitur platform dan AI di alur	Tes lisan 2, 5%
14	UI/UX, aksesibilitas, testing, dokumentasi	Tes lisan 3, 5%
15	Build rilis, gladi presentasi, laporan hasil proyek	Laporan 5%, sebelum minggu 16

Minggu 16 adalah Demo Day di jadwal UAS: tes lisan final, 35%.

Dokumen perencanaan untuk tes lisan 1

1 **Kebutuhan**
Fitur wajib dari Bank Proyek dan cerita pengguna tema kalian.

2 **Diagram arsitektur**
Paket ui, domain, dan data, dengan arah dependensinya.

3 **Pembagian kerja**
Penanggung jawab setiap fitur dan setiap sprint.

4 **Repository siap**
README, CI hijau, dan aturan branch untuk main.

Semua ditulis di README atau folder docs. Di tes lisan 1 setiap anggota menjelaskan satu bagian.

Rubrik tes lisan 1: perencanaan dan fitur utama

Kriteria	Sangat baik 85 sampai 100	Baik 70 sampai 84	Cukup 55 sampai 69	Kurang di bawah 55
Perencanaan dan repository CPMK0502, 5% Per anggota	Kebutuhan, diagram arsitektur, dan pembagian kerja tertulis di README dan dijelaskan tepat oleh setiap anggota	Dokumen lengkap, penjelasan kurang tepat di satu bagian	Dokumen sebagian, penjelasan hanya membacakan	Tidak ada dokumen perencanaan
Fitur utama berfungsi CPMK0501, 5% Kelompok	Lima layar, data REST, dan penyimpanan offline jalan saat demo	Satu dari tiga bagian itu belum jalan	Dua bagian belum jalan	Aplikasi tidak bisa didemokan

Tes Lisan (Tugas Kelompok), bobot 10% nilai akhir: CPMK0501 5%, CPMK0502 5%. Nilai kriteria dikalikan bobotnya. Kriteria per anggota dinilai lewat tanya jawab kode.

Bagian 4

Coba sendiri, lalu bawa ke proyekmu

Tiga latihan di kelas, arsitektur dan CI kelompok, checkpoint proyek, dan tiket keluar.

Tiga latihan di kelas

Latihan 1

Pindahkan ke tiga paket. Susun LaporKampus atau proyek kelompok menjadi ui, domain, dan data. Selesai kalau tidak ada `import io.ktor` di paket ui.

Latihan 2

Workflow pertama. Buat `ci.yml` dari slide tadi, push, lalu tunggu hasil hijau di tab Actions. Tambahkan badge ke README.

Latihan 3

CI merah dengan sengaja. Di branch baru, ubah satu assert agar gagal, buka pull request, lihat CI merah. Perbaiki, push lagi, minta teman me-review.

Sekitar tujuh menit per latihan, kerjakan berpasangan. Minggu ini tidak ada folder hands-on, bahannya repository kalian sendiri. Belum punya test? Salin satu test dari folder hands-on Testing dan DI.

Kalau error

Gejala atau pesan	Penyebab	Yang dicoba
Workflow tidak muncul di tab Actions	Berkas tidak di .github/workflows, atau ekstensinya bukan .yml	Periksa folder dan nama berkas
Invalid workflow file	Indentasi YAML memakai tab atau tidak sejajar	Pakai spasi, sejajarkan steps
Task 'desktopTest' not found	Nama modul atau target berbeda	<code>./gradlew tasks</code> , lalu sesuaikan
Banyak Unresolved reference setelah pindah paket	Baris package dan import belum diubah	Refactor, Move di Android Studio
Badge tidak menampilkan status	Nama berkas di alamat badge salah, atau workflow belum pernah jalan	Samakan nama berkas, push sekali lagi

Arsitektur dan CI kelompokmu

1 Tiga paket
ui, domain bila perlu, dan data, tanpa Ktor atau SQLDelight di ui.

2 CI di setiap push
Workflow build dan test, badge hijau di README.

3 Main dijaga
Aturan branch: wajib pull request, review, dan CI lulus.

4 Rencana sprint
Tabel minggu 11 sampai 16 di README dengan penanggung jawab.

CONTOH DARI LAPORKAMPUS

Paket ui, domain, data. ci.yml menjalankan assembleDebug dan desktopTest. Pull request Tambah filter status di-review Ayu sebelum masuk main.

Checkpoint proyek minggu 11

1

CI hijau

Workflow build dan test jalan di setiap push dan pull request, badge di README.

2

Arsitektur rapi

Paket ui, domain, data, dan layar tidak memanggil Ktor atau SQLDelight langsung.

3

Kerja lewat pull request

Setiap anggota punya minimal satu pull request yang di-review teman.

Dikumpulkan

Push ke repository kelompok. README memuat badge CI, struktur paket, dan rencana sprint minggu 11 sampai 16.

Dicek di awal pertemuan berikutnya: tuntas, sebagian, atau belum.

TUTUP

Tiket keluar



1 CI
Apa yang dijalankan CI setiap kali ada push?

2 Lapisan
Kenapa lapisan ui tidak boleh memanggil Ktor langsung?

3 Pull request
Apa isi pull request yang baik?

Jawab singkat di kertas atau formulir LMS sebelum keluar kelas.

TUTUP

Minggu depan: code review dan tes lisan 1



MASALAH MINGGU 12

Semalam kelompok kalian sudah latihan dan semuanya lancar. Di depan kelas, begitu tombol Kirim ditekan, aplikasi menutup sendiri. Apa yang kalian lakukan dalam 60 detik berikutnya?

Persiapan

Dokumen perencanaan di README. Lima layar, data REST, dan offline jalan di HP atau emulator. Setiap anggota siap menjelaskan bagiannya.

Terima kasih

Sampai jumpa di pertemuan 12

Ada pertanyaan? Sampaikan sekarang, lewat LMS, atau email ke muhammad.algifari@if.itera.ac.id